

Lecture 3. Maps

PUBH 6199: Visualizing Data with R, Summer 2026

Xindi (Cindy) Hu, ScD

2026-06-02



Time to start thinking about your final project



Time to start thinking about your final project



Time to start thinking about your final project

Be prepared to share your project idea via a 2-min pitch in lab 4 next week (June 11th).

- Project idea
- Research questions
- Potential data sources
- Format preference and anticipated visualizations

See [course website](#) for project details and deadlines.



Outline for today

- **Types of maps for spatial data**
- Map design considerations (color, projection, legend)
- R packages for mapping
- Static maps with `tmap`
- Interactive maps with `leaflet`
- Accessing spatial data with `tigris` and `tidycensus`



Types of Maps for Spatial Data

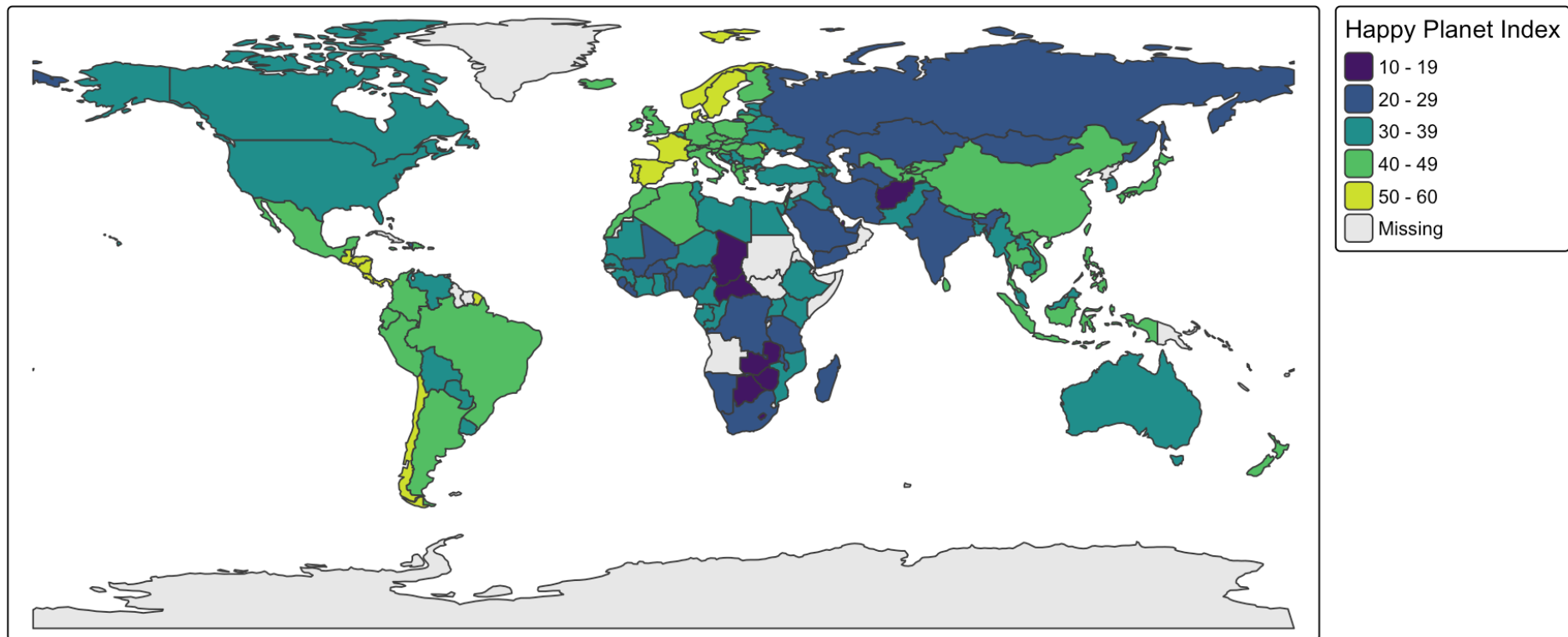
1. Choropleth Map

- Display the spatial distribution of a variable across divided geographical areas
- Best for **normalized** data (e.g., rates)
- Variable encoding: **color** (intensity or hue)



Example: Choropleth

```
1 library(tmap)
2 data("World")
3 tm_shape(World) +
4   tm_polygons("HPI", palette = "viridis", title = "Happy Planet Index")
```



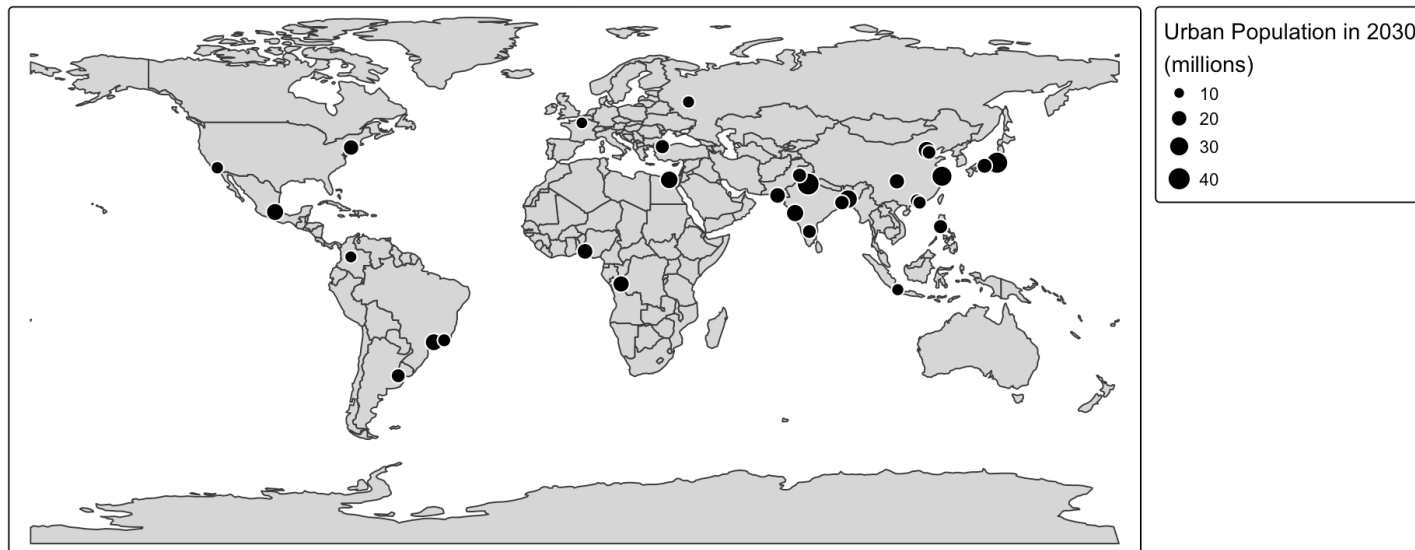
2. Point Map

- Each point represents a location
- Can show locations of events, facilities, cases
- Variable encoding: **color**, **shape**, **size**



Example: Point Map

```
1 library(tidyverse)
2 library(spData)
3 library(tmap)
4 data(urban_agglomerations)
5 urb_2030 <- urban_agglomerations |> filter(year == 2030)
6 tm_shape(World) +
7   tm_polygons() +
8   tm_shape(urb_2030) +
9   tm_symbols(fill = "black", col = "white", size = "population_millions",
10             size.legend = tm_legend(title = "Urban Population in 2030\n(millions)"))
```



3. Heat Map (Density)

- Visualizes concentration of events
- Uses kernel density estimation or hex bins
- Variable encoding: **color** (intensity)

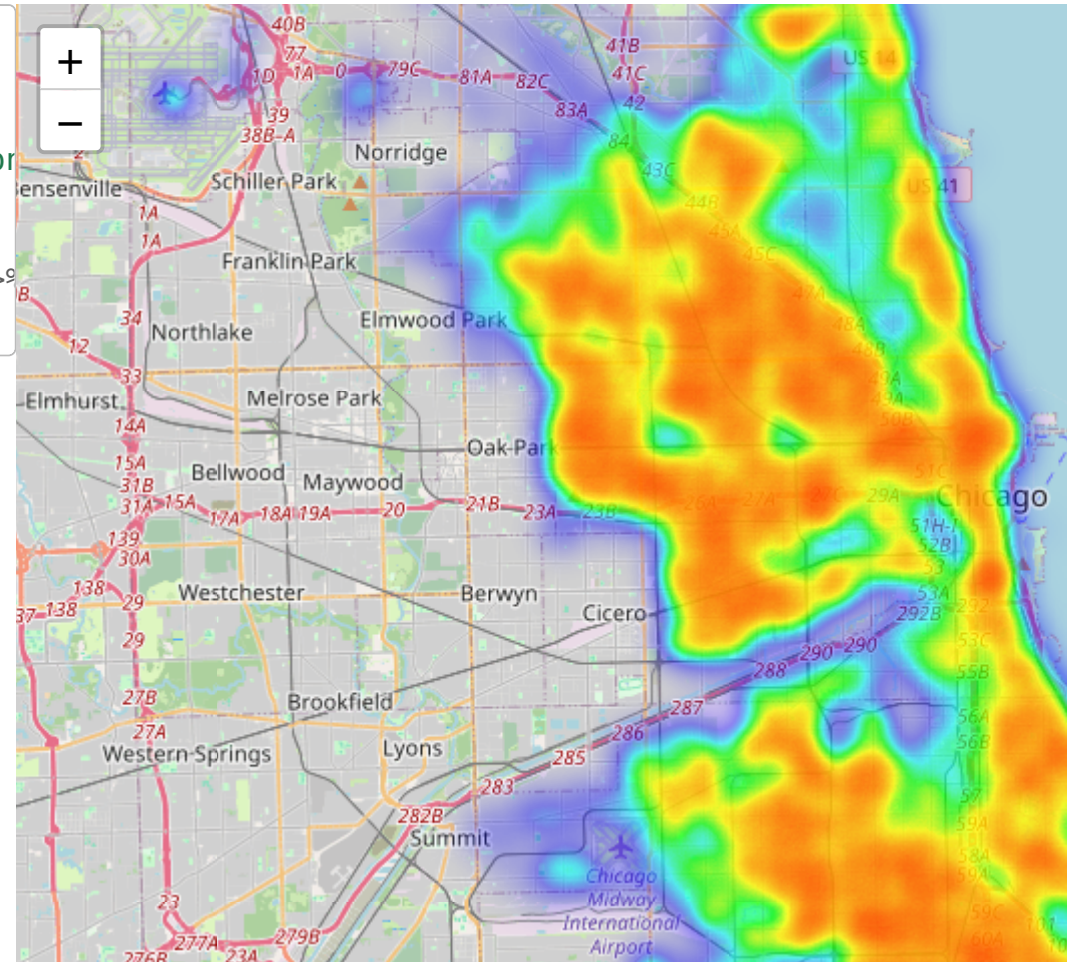


Example: Heat map with `leaflet.extras`

```

1 library(leaflet)
2 library(leaflet.extras)
3 thefts_coords <- read_csv("data/thefts_coords.csv")
4 thefts_sf <- st_as_sf(thefts_coords, coords = c("lon", "lat"))
5 leaflet(thefts_sf) %>%
6   addProviderTiles("OpenStreetMap") %>%
7   addHeatmap(blur = 15, max = 0.05, radius = 10) %>%
8   setView(lng = -87.6298, lat = 41.8781, zoom = 11)

```



4. Faceted Map

- Create small multiples to compare across categories
- Great for time series or group comparisons

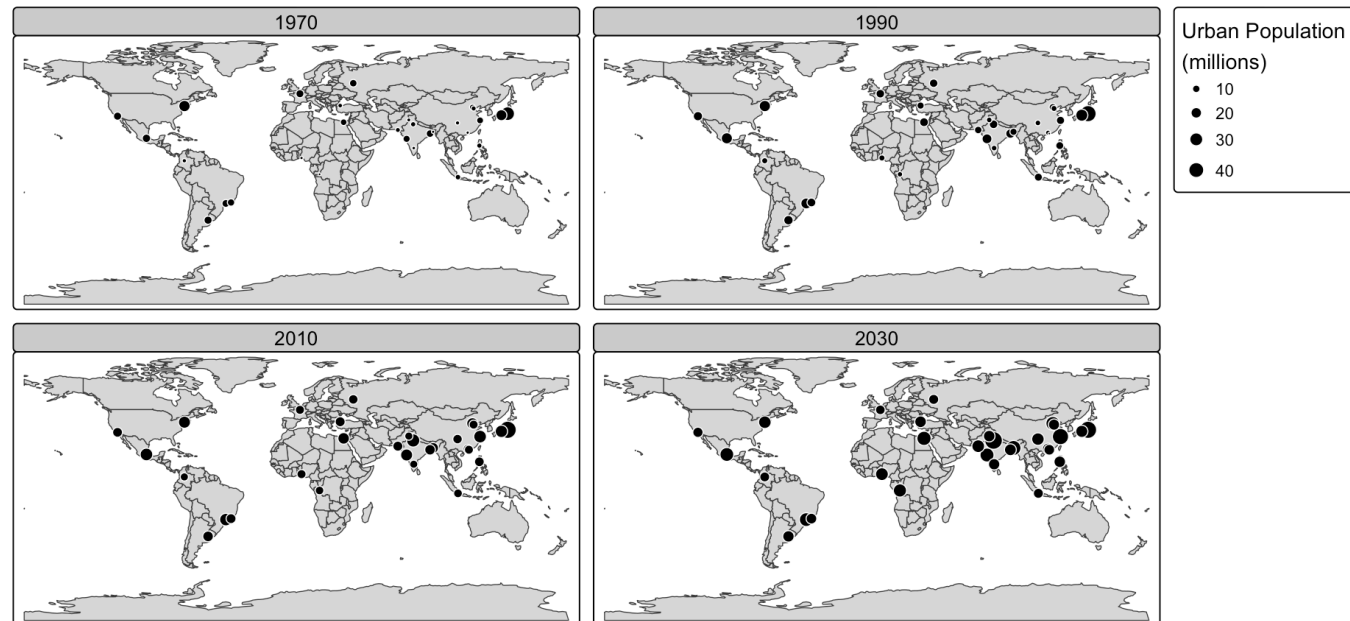


Example: Faceted Map

```

1 library(tidyverse)
2 library(spData)
3 library(tmap)
4 data(urban_agglomerations)
5 urb_1970_2030 <- urban_agglomerations |> filter(year %in% c(1970, 1990, 2010, 2030))
6 tm_shape(World) +
7   tm_polygons() +
8   tm_shape(urb_1970_2030) +
9   tm_symbols(fill = "black", col = "white", size = "population_millions",
10             size.legend = tm_legend(title = "Urban Population\n(millions)")) +
11   tm_facets(by = "year", ncol = 2)

```

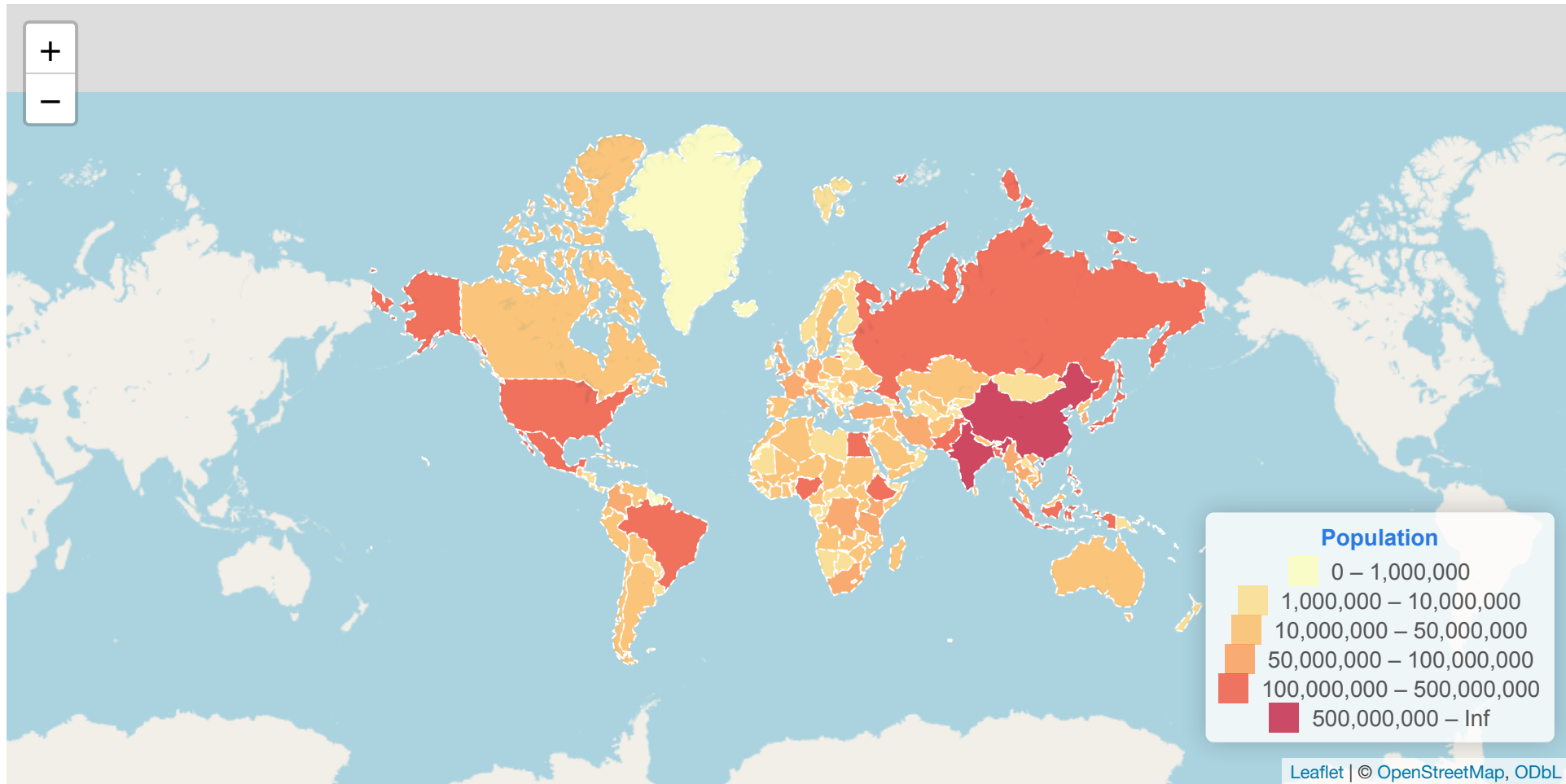


5. Interactive Map

- Enable zooming, hovering, filtering
- Useful for dashboards, web apps



Example: Interactive Map



Outline for today

- Types of maps for spatial data
- **Map design considerations (color, scale, projection, legend)**
- R packages for mapping
- Static maps with `tmap`
- Interactive maps with `leaflet`
- Accessing spatial data with `tigris` and `tidycensus`



Color Choices for Maps

- **Sequential** palettes for ordered values
- **Diverging** palettes for above/below mean
- **Qualitative** for categories

QUANTITATIVE COLOR SCALES

SEQUENTIAL & UNCLASSED



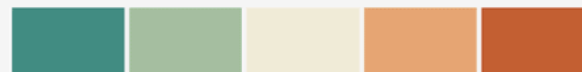
SEQUENTIAL & CLASSED



DIVERGING & UNCLASSED



DIVERGING & CLASSED



QUALITATIVE COLOR SCALES

CATEGORICAL



Source: [Which color scale to use when visualizing data](#), by Lisa Charlotte Muth.

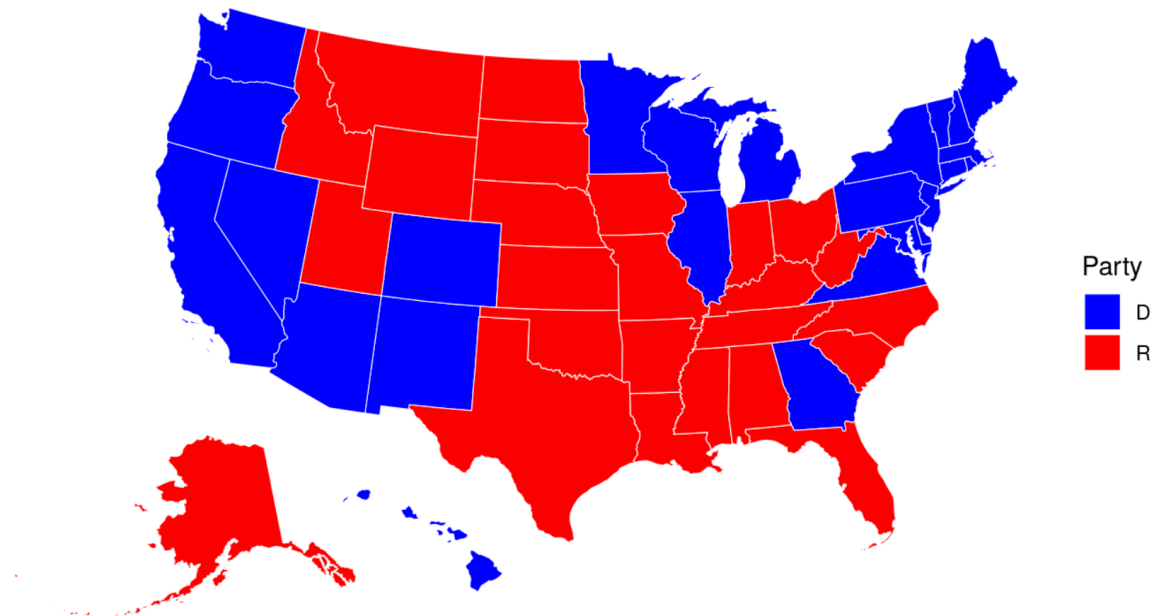
Categorical scales

- Use distinct **hues** for different categories
- Limit to **no more than 7 hues**

QUALITATIVE COLOR SCALES



2020 US presidential election results by state



Note: Nebraska and Maine split electoral college votes by congressional district

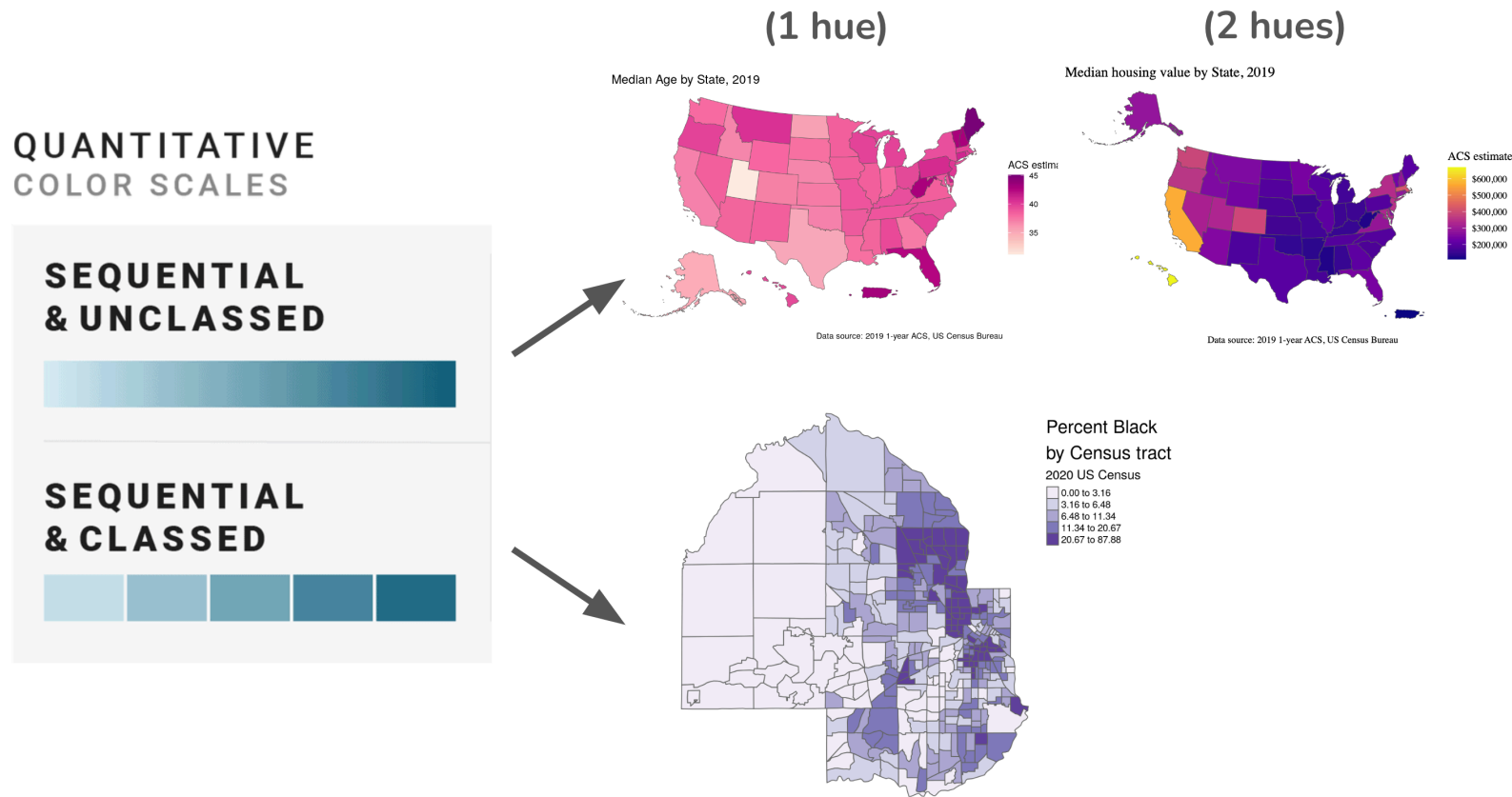
Source: [Analyzing US Census Data](#), by Kyle Walker

PUBH 6199: Visualizing Data with R



Sequential Scales

- Map value to color on a continuum, based on both **intensity** and **hue**
- Use for ordered data (e.g., population, income)



Source: [Analyzing US Census Data](#), by Kyle Walker

PUBH 6199: Visualizing Data with R



Diverging Scales

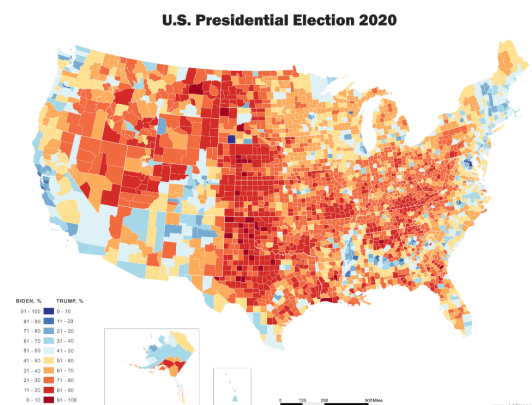
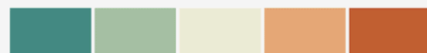
- Use for data with a **meaningful midpoint** (e.g., above/below average)
- Two contrasting colors with a **neutral midpoint** (e.g., white/light gray)

QUANTITATIVE COLOR SCALES

DIVERGING & UNCLASSED



DIVERGING & CLASSED

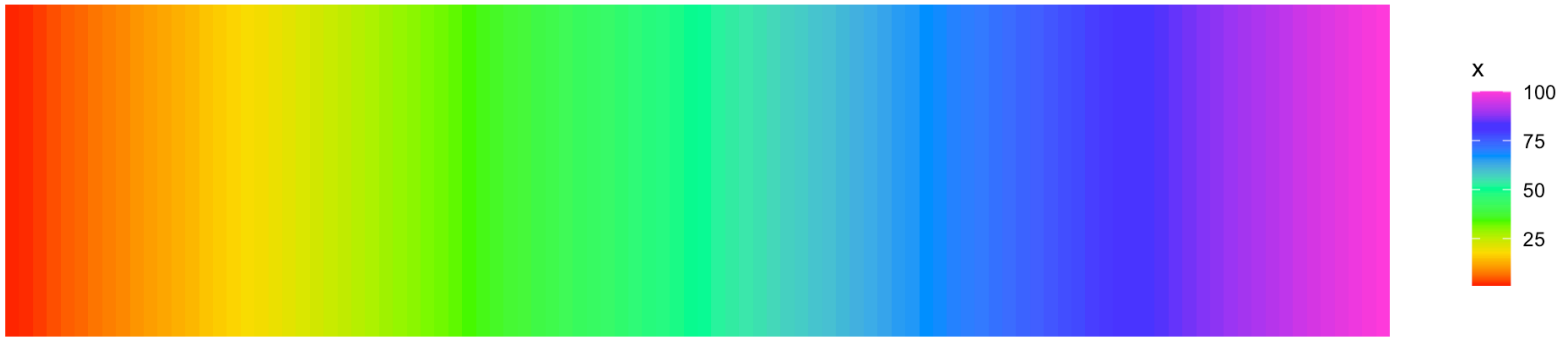


Source: [2020 U.S. Election Mapped](#), by Vivid Maps

Avoid Misleading Colors

- Don't use rainbow: not perceptually uniform

Rainbow (Perceptually Nonlinear)



Viridis (Perceptually Uniform)



- Consider accessibility (color-blind safe palettes)
- Avoid encoding meaning with non-intuitive colors

Map Projections

- A projection distorts shape, area, distance, or direction
- Use equal-area projections for choropleths

Why all world maps are wrong

Vox



Watch on

Common Projections in R

Use Case	Recommended Projection	EPSG Code
Equal-area choropleths	Albers Equal Area	5070
Interactive maps	Web Mercator	3857
Global perspective	Robinson or Winkel Tripel	54030 / 54042
Local detail (U.S.)	NAD83 / State Plane	varies

Use `st_transform()` to convert:

```
1 my_data <- sf::st_transform(my_data, crs = 5070)
```

In `tmap`:

- In static mode: all layers are reprojected to match the first layer.
- In interactive mode: all layers are projected to EPSG:3857.

Outline for today

- Types of maps for spatial data
- Map design considerations (color, scale, projection, legend)
- **R packages for mapping**
- Static maps with `tmap`
- Interactive maps with `leaflet`
- Accessing spatial data with `tigris` and `tidycensus`



R Ecosystem for Mapping

- **Data handling:** `sf`, `sp`
- **Thematic mapping:** `tmap`, `ggplot2`, `cartography`
- **Basemaps & interactivity:** `leaflet`, `mapview`, `ggmap`
- **Shapefiles:** `rgdal`, `rmapshaper`
- **Data access:** `tigris`, `tidycensus`



{sf}: simple features

The `{sf}` package is the **standard way to work with vector spatial data in R**. It replaces older tools like `{sp}` with a **simple, tidy-friendly** interface.

Key Features of `{sf}`

- Stores **geometry + attributes** in a single `data.frame`-like object
- Built on **simple features** standard (ISO 19125-1)
- Fully compatible with **dplyr**, **ggplot2**, **tmap**
- Uses `sfc` column to store spatial information (e.g., points, polygons)

Simple feature collection with 100 features and 6 fields
 Geometry type: MULTIPOLYGON
 Dimension: XY
 Bounding box: xmin: -84.32385 ymin: 33.88199 xmax: -75.45698 ymax: 36.58965
 Geodetic CRS: NAD27
 First 3 features:

	BIR74	SID74	NWBIR74	BIR79	SID79	NWBIR79	geometry
1	1091	1	10	1364	0	19	MULTIPOLYGON (((-81.47276 3...
2	487	0	10	542	3	12	MULTIPOLYGON (((-81.23989 3...
3	3188	5	208	3616	6	260	MULTIPOLYGON (((-80.45634 3...

Simple feature
 Simple feature geometry list-column (sfc)
 Simple feature geometry (sfg)



Outline for today

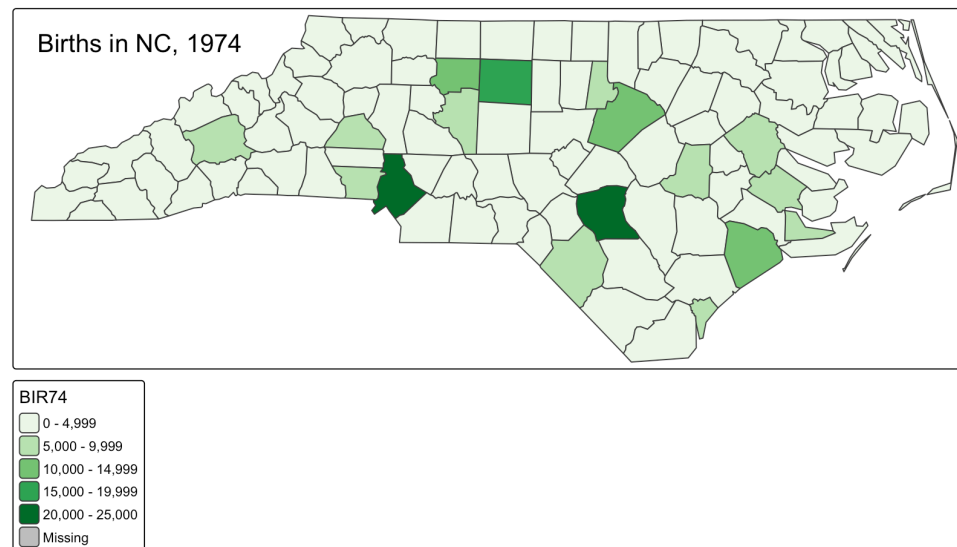
- Types of maps for spatial data
- Map design considerations (color, scale, projection, legend)
- R packages for mapping
- **Static maps with `tmap`**
- Interactive maps with `leaflet`
- Accessing spatial data with `tigris` and `tidycensus`



Static Mapping with **tmap**

- Similar to `ggplot2`, based on “the grammar of graphics”
- Supports both static and interactive modes
- Excellent for quick, polished maps, sensible defaults

```
1 library(tmap)
2 tmap_mode("plot")
3 nc <- st_read("data/nc.shp", quiet = TRUE) # nc is an `sf` object
4 tm_shape(nc) + # defines input data
5   tm_polygons("BIR74", palette = "Greens") + # mapping data to aesthetics
6   tm_layout(title = "Births in NC, 1974")
```



How {tmap} works?

{tmap} adopts an intuitive approach to map-making: the addition operator + adds a new layer, followed by tm_*():

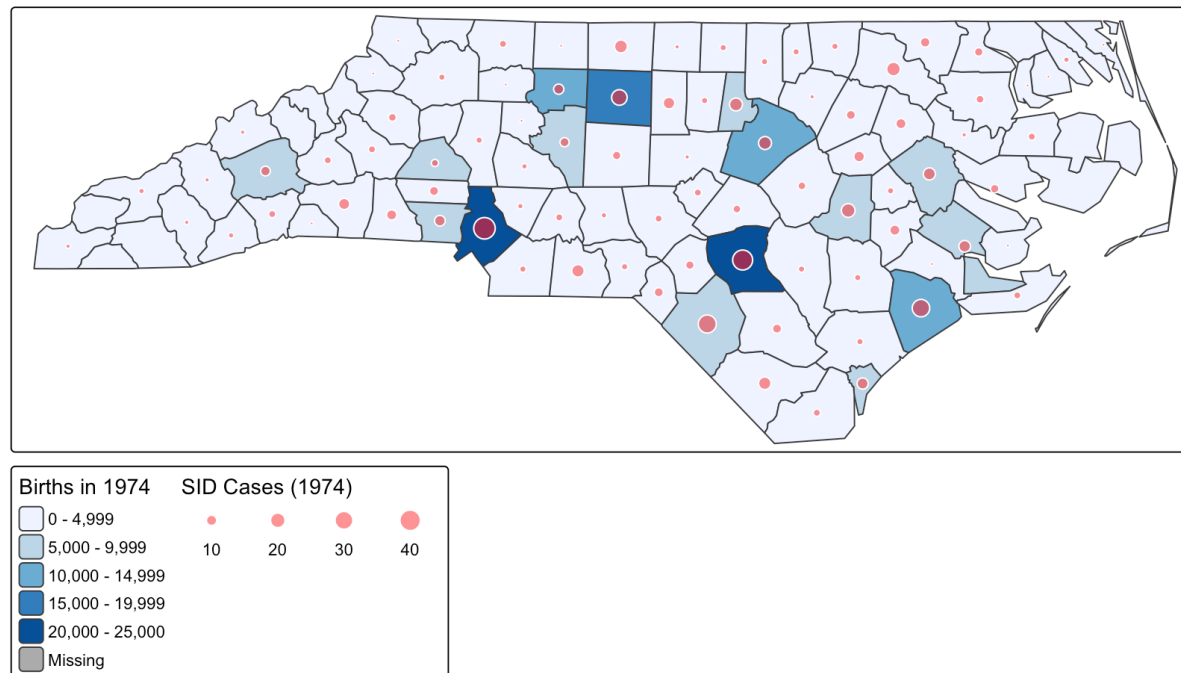
- `tm_fill()`: shaded areas for (multi)polygons
- `tm_borders()`: border outlines for (multi)polygons
- `tm_polygons()`: both, shaded areas and border outlines for (multi)polygons
- `tm_lines()`: lines for (multi)linestrings
- `tm_symbols()`: symbols for (multi)points, (multi)linestrings, and (multi)polygons
- `tm_raster()`: colored cells of raster data (there is also `tm_rgb()` for rasters with three layers)
- `tm_text()`: text information for (multi)points, (multi)linestrings, and (multi)polygons



Adding layers in {tmap}

- `tm_polygons()`: for choropleth maps
- `tm_symbols()`: for point data, size and color can represent different variables

```
1 # Create the map: choropleth + bubbles
2 tm_shape(nc) +
3   tm_polygons("BIR74", palette = "brewer.blues", title = "Births in 1974") +
4   tm_symbols(size = "SID74", col = "red", alpha = 0.5, border.col = "white",
5             title.size = "SID Cases (1974)")
```



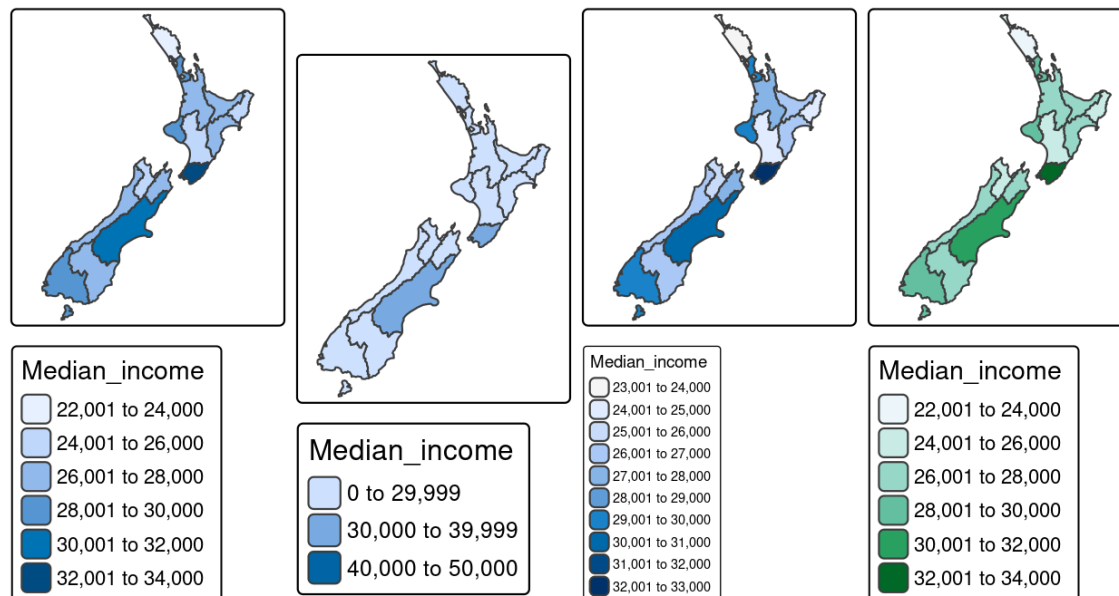
Scale

Scales control how the values are represented on the map and in the legend, and can have a major impact on how spatial variability is portrayed

```

1 tm_shape(nz) + tm_polygons(fill = "Median_income")
2 tm_shape(nz) + tm_polygons(fill = "Median_income",
3                             fill.scale = tm_scale(breaks = c(0, 30000, 40000, 50000)))
4 tm_shape(nz) + tm_polygons(fill = "Median_income",
5                             fill.scale = tm_scale(n = 10))
6 tm_shape(nz) + tm_polygons(fill = "Median_income",
7                             fill.scale = tm_scale(values = "BuGn"))

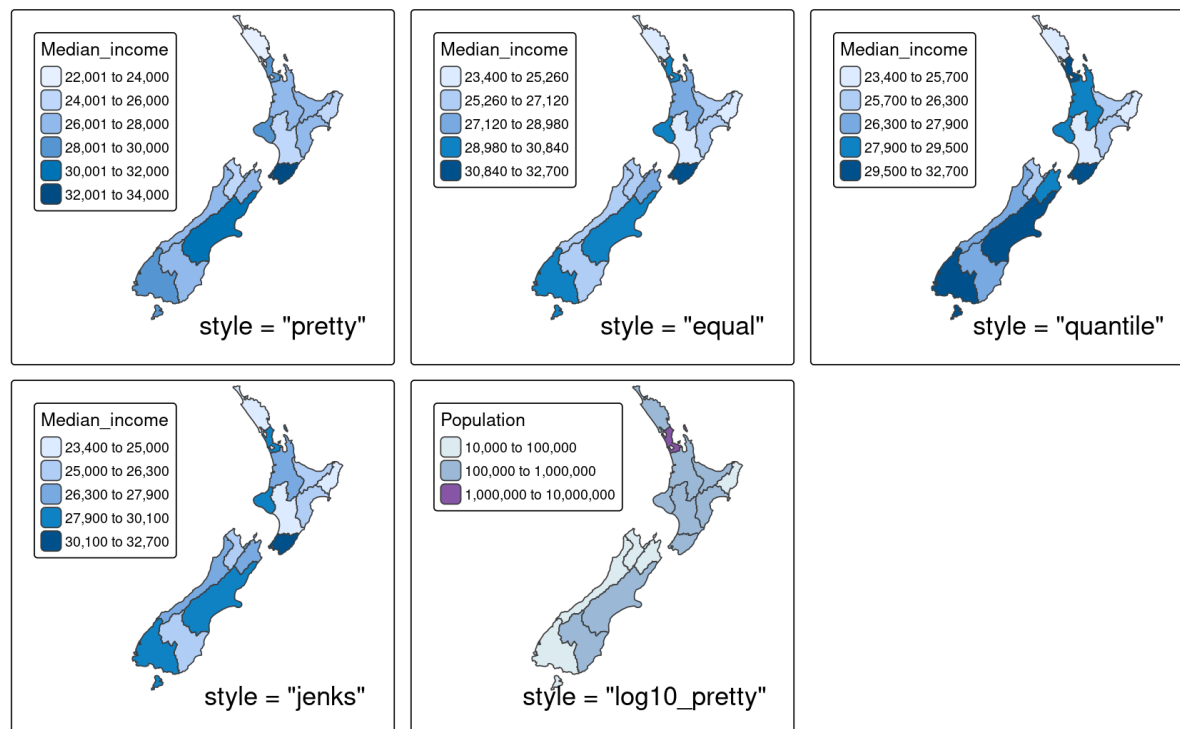
```



Style options for classifying map data

`tm_scale_intervals(style = "pretty"):`

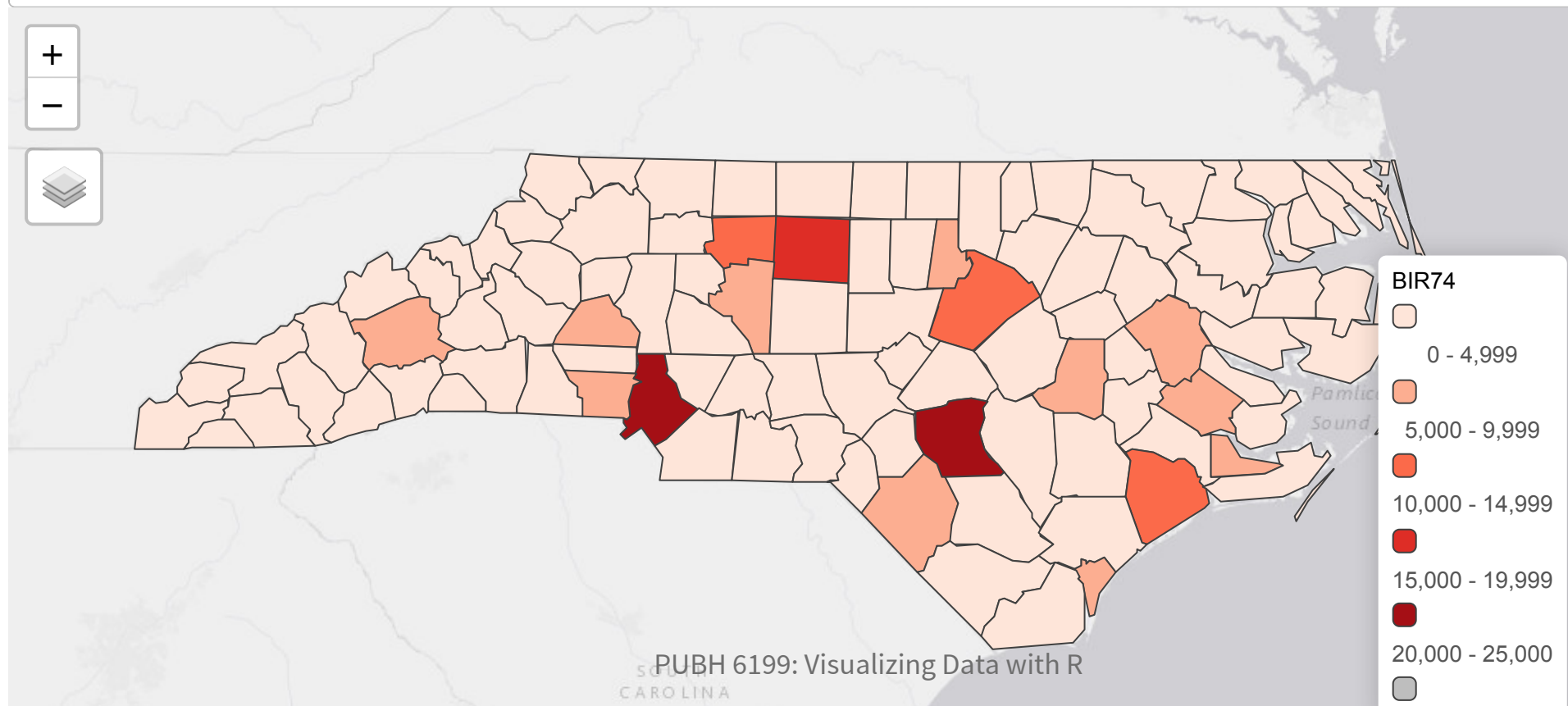
- **“pretty”**: Rounded, evenly spaced breaks (**default**).
- **“equal”**: Equal-width bins; poor fit for skewed data — may hide variation.
- **“quantile”**: Equal count per bin; be careful with wide bin ranges.
- **“jenks”**: Optimizes natural groupings; can be slow with large datasets.
- **“log10_pretty”**: Log-scaled breaks; only appropriate for right-skewed, positive values.



Switch to interactive {tmap}

A unique feature of {tmap} is its ability to create static and interactive maps using the same code. Maps can be viewed interactively at any point by switching to view mode, using the command `tmap_mode("view")`

```
1 tmap_mode("view")
2 tm_shape(nc) +
3   tm_polygons("BIR74", palette = "brewer.reds")
```



Outline for today

- Types of maps for spatial data
- Map design considerations (color, scale, projection, legend)
- R packages for mapping
- Static maps with `tmap`
- **Interactive maps with `leaflet`**
- Accessing spatial data with `tigris` and `tidycensus`



Interactive Mapping with `{leaflet}` in R

- `{leaflet}` is the most widely used interactive mapping package in R.
- It provides a relatively low-level interface to the Leaflet.js JavaScript library leafletjs.com.
- Maps start with `leaflet()` and use pipeable layers like `addTiles()`, `addCircles()`, and `addPolygons()`.

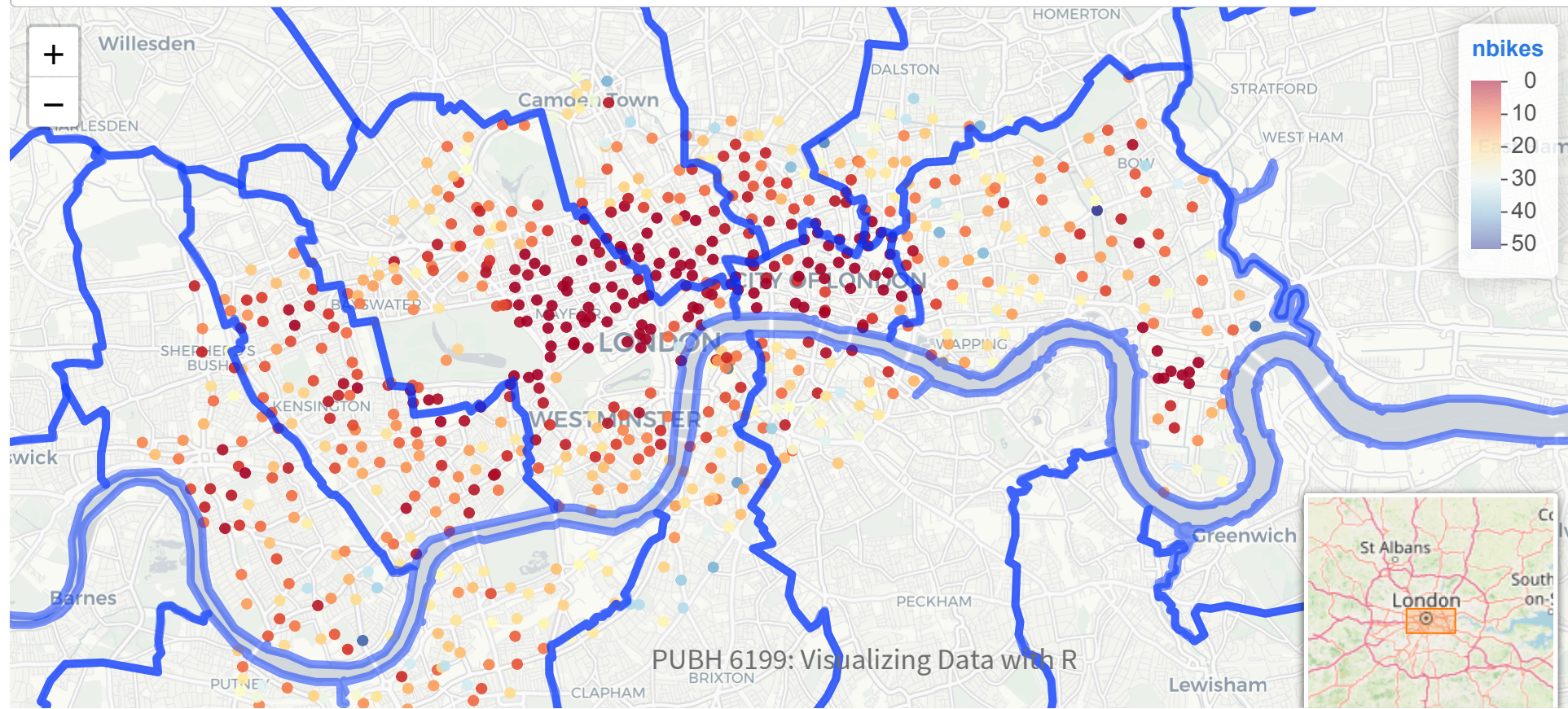


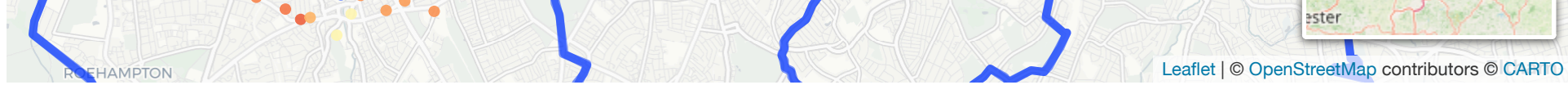
Example `leaflet` Map

```

1 pal <- colorNumeric("RdYlBu", domain = cycle_hire$nbikes)
2 # cycle_hire is an `sf` object with columns: name, nbikes, geometry, built-in data from spData
3 leaflet(cycle_hire) |>
4   addProviderTiles(providers$CartoDB.Positron) |>
5   addCircles(col = ~pal(nbikes), opacity = 0.9) |>
6   # lnd is a London boroughs shapefile
7   addPolygons(data = lnd, fill = FALSE) |>
8   addLegend(pal = pal, values = ~nbikes) |>
9   setView(lng = -0.1, lat = 51.5, zoom = 12) |>
10  addMiniMap()

```





Outline for today

- Types of maps for spatial data
- Map design considerations (color, scale, projection, legend)
- R packages for mapping
- Static maps with `tmap`
- Interactive maps with `leaflet`
- **Accessing spatial data with `tigris` and `tidycensus`**



Getting Data with **tigris**

The `{tigris}` package provides access to U.S. Census Bureau geographic data. Shapefiles downloaded using `{tigris}` will be loaded as a simple features (sf) object with geometries.



- A **shapefile** is a vector data file format commonly used for geospatial analysis.
- Shapefiles contain information for spatially describing features (e.g. points, lines, polygons), as well as any associated attribute information.
- You can find / download shapefiles online (e.g. from the US Census Bureau), or depending on the tools available, access them via packages (like we're doing today).

Getting U.S. County Shapefiles

Entire US

```
1 library(tigris)
2 library(sf)
3 counties <- counties(state = NULL, cb = TRUE, progress = FALSE)
4 # Use `cb = TRUE` for simplified geometries
5 glimpse(counties)
```

Rows: 3,235

Columns: 13

```
$ STATEFP    <chr> "01", "01", "01", "10", "01", "01",
"04", "05", "05", "05",...
$ COUNTYFP   <chr> "069", "023", "113", "005", "071",
"089", "015", "017", "12...
$ COUNTYNS   <chr> "00161560", "00161537", "00161583",
"00217269", "00161561",...
$ GEOIDFQ    <chr> "05000000US01069", "05000000US01023",
"05000000US01113", "0500...
$ GEOID      <chr> "01069", "01023", "01113", "10005",
"01071", "01089", "0401...
$ NAME       <chr> "Houston", "Choctaw", "Russell",
"Sussex", "Jackson", "Madi...
$ NAMELSAD   <chr> "Houston County", "Choctaw County",
"Russell County", "Suss...
$ STUSPS     <chr> "AL", "AL", "AL", "DE", "AL", "AL",
"AZ", "AR", "AR", "AR",...
$ STATE_NAME <chr> "Alabama", "Alabama", "Alabama",
"Delaware", "Alabama", "Al...
```

One state

```
1 library(tigris)
2 library(sf)
3 counties_md <- counties(state = "Maryland", cb = TRUE, progress = FALSE)
4 # Use `cb = TRUE` for simplified geometries
5 glimpse(counties_md)
```

Rows: 24

Columns: 13

```
$ STATEFP    <chr> "24", "24", "24", "24", "24", "24",
"24", "24", "24", "24",...
$ COUNTYFP   <chr> "005", "019", "017", "015", "041",
"037", "039", "011", "03...
$ COUNTYNS   <chr> "01695314", "00596495", "01676992",
"00596115", "00592947",...
$ GEOIDFQ    <chr> "05000000US24005", "05000000US24019",
"05000000US24017", "0500...
$ GEOID      <chr> "24005", "24019", "24017", "24015",
"24041", "24037", "2403...
$ NAME       <chr> "Baltimore", "Dorchester",
"Charles", "Cecil", "Talbot", "S...
$ NAMELSAD   <chr> "Baltimore County", "Dorchester
County", "Charles County", ...
$ STUSPS     <chr> "MD", "MD", "MD", "MD", "MD", "MD",
"MD", "MD", "MD", "MD",...
$ STATE_NAME <chr> "Maryland", "Maryland", "Maryland",
"Maryland", "Maryland",...
```

Getting Census Data with **tidycensus**

```
1 library(tidycensus)
2 # add census api key
3 invisible(
4   census_api_key(Sys.getenv("CENSUS_API_KEY"), install = TRUE, overwrite = TRUE)
5 )
6 options(tigris_use_cache = TRUE)
7 income_md <- get_acs(geography = "county",
8   state = "MD",
9   variables = "B19013_001",
10  geometry = FALSE,
11  show_progress = FALSE)
```

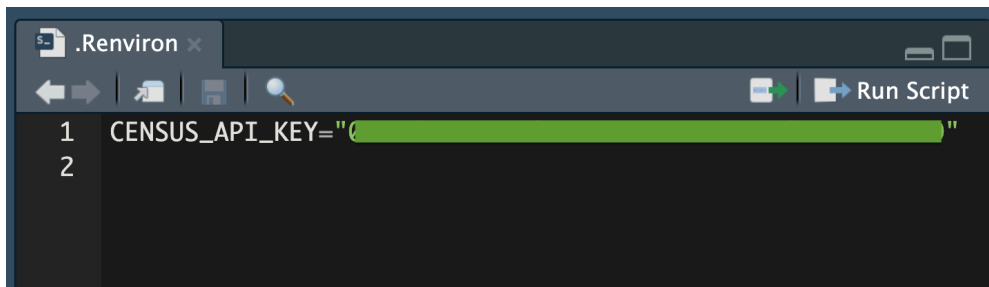
Follow [this tutorial](#) on how to get started with `{tidycensus}` packages. Remember to add your `.Renvirom` to `.gitignore` so that you do not share your API keys.



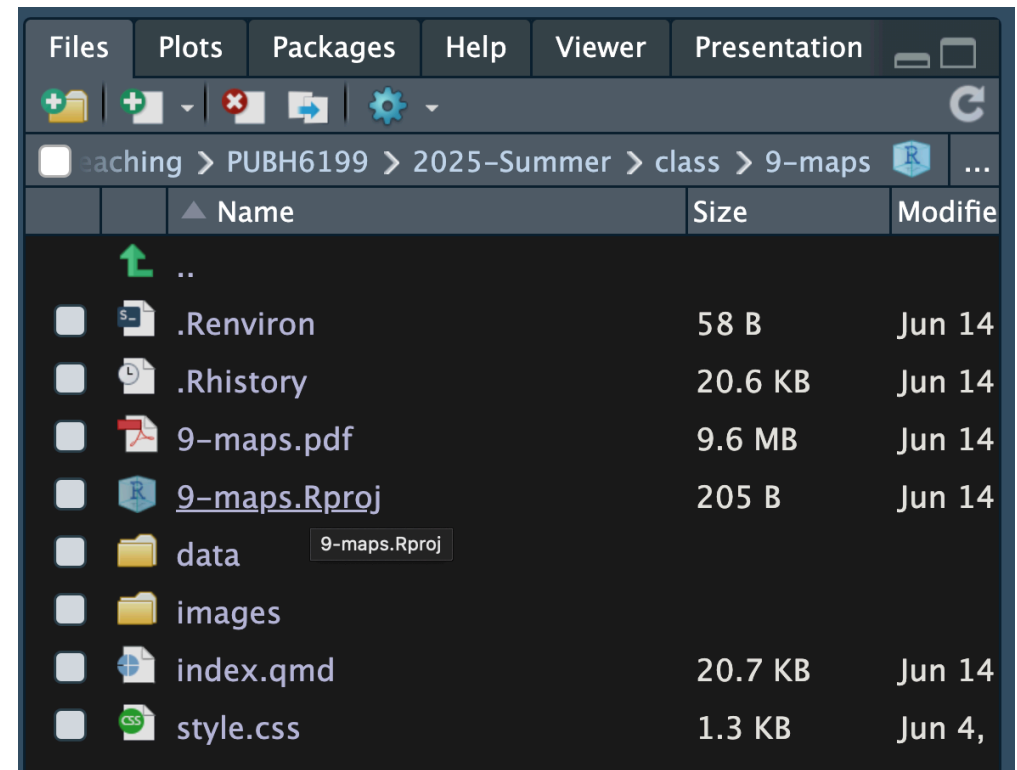
How to store API key without committing it to GitHub

To use the `{tidycensus}` package, you need to set up a Census API key. You can obtain one from the [Census Bureau's website](#).

You can store your API key in a `.Renviron` file in your repository's root directory (same directory as your `.Rproj`). Add `.Renviron` to `.gitignore` so that this file is not committed to GitHub.



A screenshot of an R script editor window titled ".Renviron". The editor shows two lines of code: line 1 is `CENSUS_API_KEY="` and line 2 is `"`. The text between the quotes is highlighted in green. A "Run Script" button is visible in the top right corner of the editor.



A screenshot of the RStudio file explorer showing the project structure. The breadcrumb path is `teaching > PUBH6199 > 2025-Summer > class > 9-maps`. The file list includes:

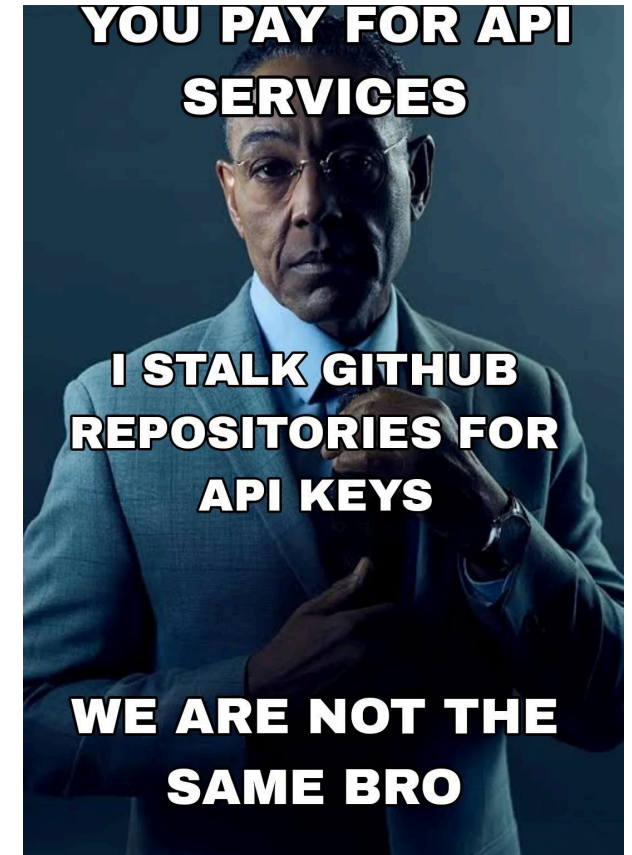
	Name	Size	Modified
↑	..		
📄	.Renviron	58 B	Jun 14
📄	.Rhistory	20.6 KB	Jun 14
📄	9-maps.pdf	9.6 MB	Jun 14
📄	9-maps.Rproj	205 B	Jun 14
📁	data		
📁	images		
📄	index.qmd	20.7 KB	Jun 14
📄	style.css	1.3 KB	Jun 4,

How to store API key without committing it to GitHub

Then in your R scripts, you can access the API key using `Sys.getenv("CENSUS_API_KEY")`. This way, you can use the API key without hardcoding it into your scripts.

```
1 library(tidycensus)
2 invisible(
3   census_api_key(Sys.getenv("CENSUS_API_KEY"), install = TRUE,
4 )
5 options(tigris_use_cache = TRUE)
```

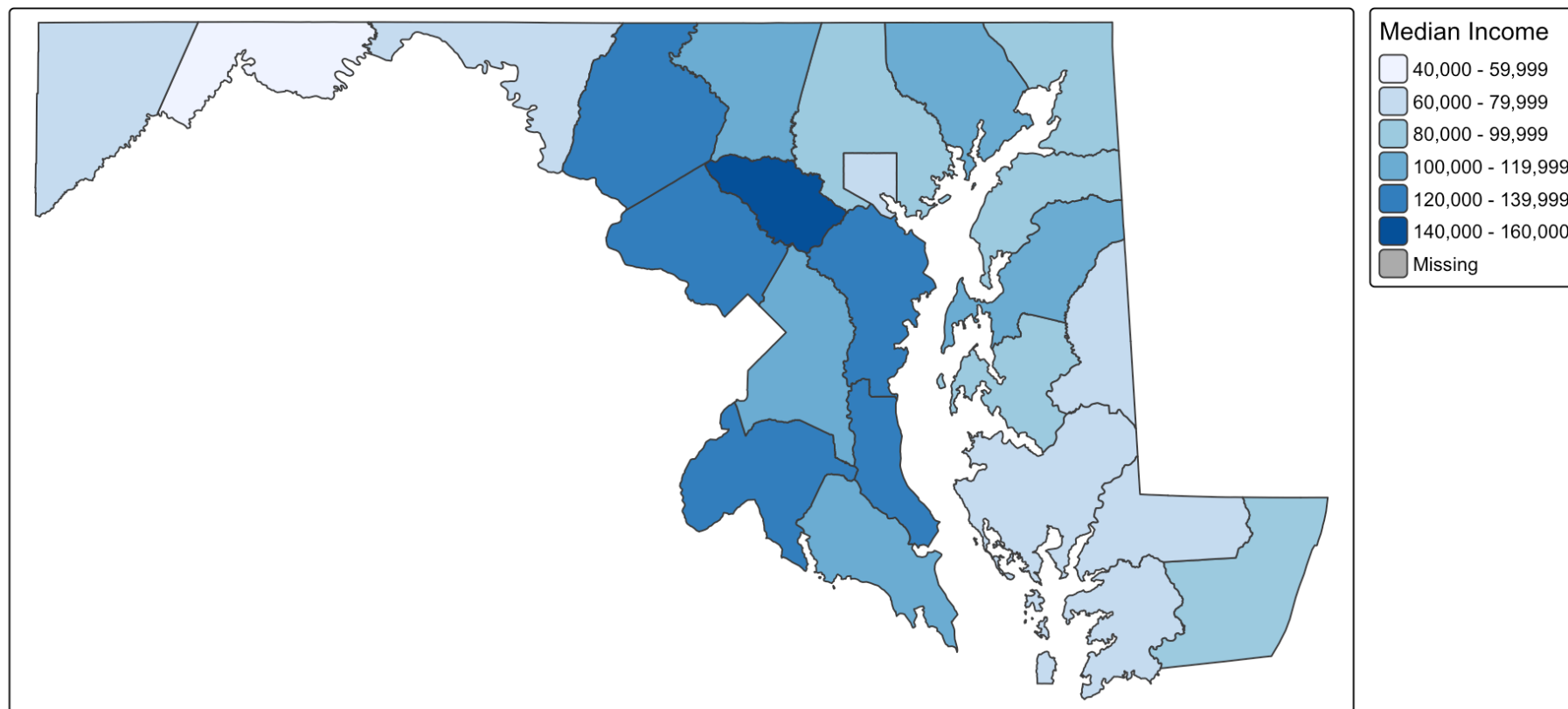
Protect yourself against this!

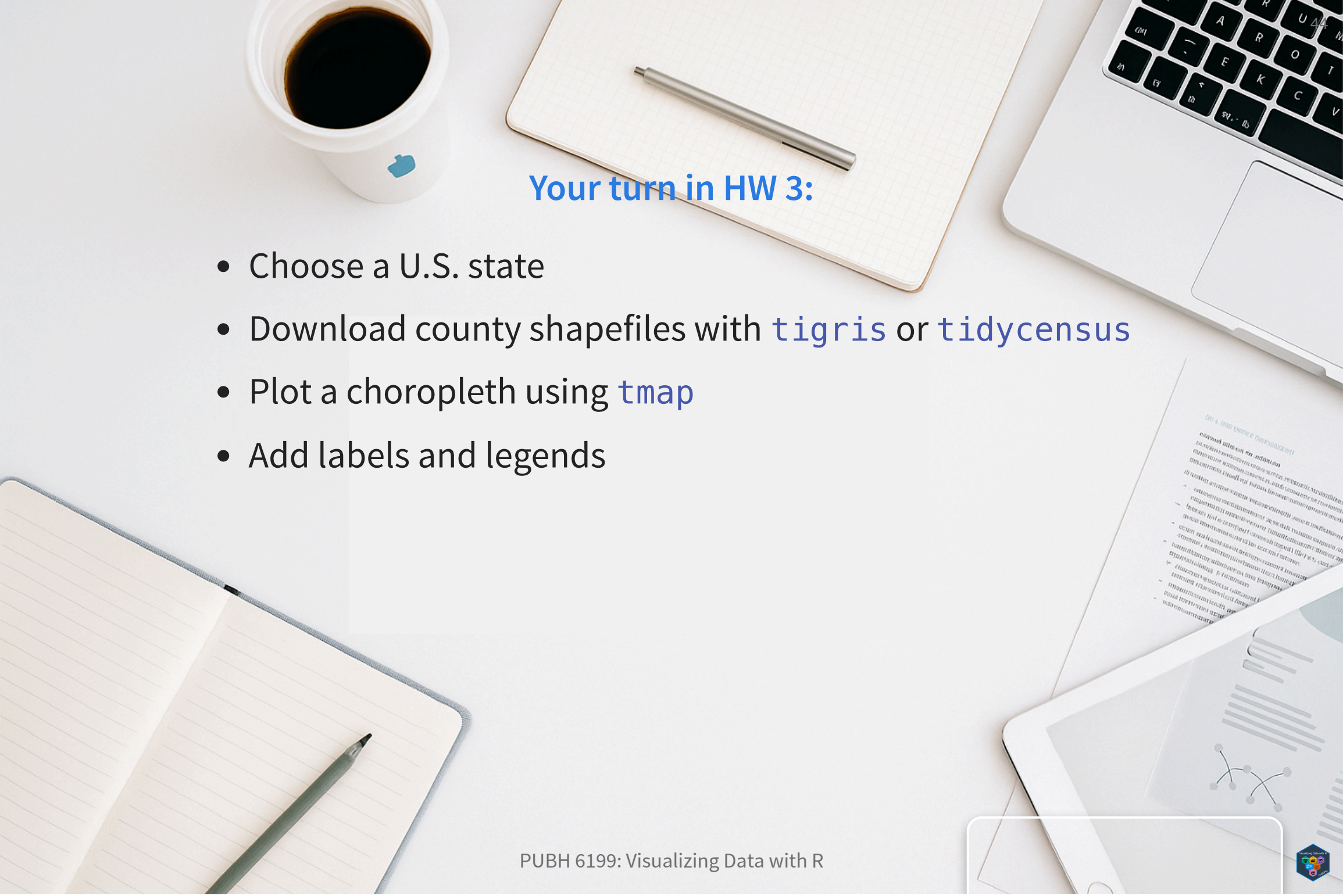


Source: [Reddit](#)

Plotting Census Data

```
1 tmap_mode("plot")
2 income_md <- counties_md |>
3   left_join(income_md, by = c("GEOID" = "GEOID")) # join the income data to the counties
4 tm_shape(income_md) +
5   tm_polygons("estimate", palette = "Blues", title = "Median Income")
```





Your turn in HW 3:

- Choose a U.S. state
- Download county shapefiles with `tigris` or `tidycensus`
- Plot a choropleth using `tmap`
- Add labels and legends

Summary

- Choose the right map for the data and audience
- Make thoughtful color and projection choices
- Use `tmap` for quick static/interactive maps
- Use `leaflet` for rich interactivity
- Access geographic data via `tigris` and `tidycensus`



End-of-Class Survey

 Fill out the end-of-class survey

~ This is the end of Lecture 3 ~